Audix ROS 2 Bridge

This workspace turns the final working Audix files into a ROS-facing system without breaking the proven controller:

- `current files/pi_master.py` stays as the reference non-ROS mission controller.
- `current files/esp_correct_pid_pi.ino` stays as the final ESP32 motor/odometry/PID firmware.
- `ros2_ws/src/audix_robot` adds a Raspberry Pi ROS 2 node that speaks the same ESP UART JSON protocol.
- `ros2_ws/src/audix_interfaces` adds typed ROS messages and services for robot commands and telemetry.

Why This Design

The ESP32 is already a stable real-time-ish motion controller. It owns wheel PID, odometry, heading hold, and position moves. The Raspberry Pi should be the ROS computer.

For this stage, the ROS 2 node runs on the Pi and bridges ROS to the final ESP firmware over UART. This follows a clean robot architecture:

- ROS 2 node on the Pi exposes services, topics, logs, and launch files.
- ESP32 firmware keeps the low-level motor loop deterministic.
- GPIO IR sensors are read by the Pi node and published as ROS messages.
- The old UART command contract remains the hardware interface.

Native micro-ROS on the ESP32 is a later firmware replacement, not something to run on the same serial port at the same time as `esp_correct_pid_pi.ino`. If the ESP becomes a micro-ROS node later, the UART text protocol should be removed and replaced by a micro-ROS Agent transport.

New Files

```
ros2_ws/
  README_AUDIX_ROS.md
  src/
    audix_interfaces/
      msg/EspTelemetry.msg
      msg/IrState.msg
      srv/Move.srv
      srv/TwistCommand.srv
      srv/RawCommand.srv
    audix_robot/
      audix_robot/esp_uart_bridge_node.py
      launch/audix_bridge.launch.py
```

ROS Graph

Node:

```
/audix/esp_uart_bridge
```

Publishers:

```
/audix/esp/telemetry      audix_interfaces/msg/EspTelemetry
/audix/esp/telemetry_raw  std_msgs/msg/String
/audix/odom               nav_msgs/msg/Odometry
/audix/ir/state           audix_interfaces/msg/IrState
```

Services:

```
/audix/esp/ping            std_srvs/srv/Trigger
/audix/esp/init_imu        std_srvs/srv/Trigger
/audix/esp/reset_odom      std_srvs/srv/Trigger
/audix/esp/reset_encoders  std_srvs/srv/Trigger
/audix/esp/stop            std_srvs/srv/Trigger
/audix/move                audix_interfaces/srv/Move
/audix/twist               audix_interfaces/srv/TwistCommand
/audix/esp/raw_command     audix_interfaces/srv/RawCommand
```

Build On The Raspberry Pi

Assuming ROS 2 is installed:

```
cd ~/pi-testing/ros2_ws
source /opt/ros/humble/setup.bash
rosdep update
rosdep install --from-paths src --ignore-src -y
colcon build --symlink-install
source install/setup.bash
```

If you use ROS 2 Jazzy instead of Humble, source Jazzy:

```
source /opt/ros/jazzy/setup.bash
```

Run

Use the real UART and real IR sensors:

```
ros2 launch audix_robot audix_bridge.launch.py port:=/dev/ttyAMA0
```

Use USB serial if the ESP32 is connected over USB:

```
ros2 launch audix_robot audix_bridge.launch.py port:=/dev/ttyUSB0
```

Run without GPIO IR hardware:

```
ros2 launch audix_robot audix_bridge.launch.py port:=/dev/ttyAMA0 mock_ir:=true
```

Run without auto IMU init or odom reset:

```
ros2 launch audix_robot audix_bridge.launch.py init_imu_on_start:=false
reset_odom_on_start:=false
```

Control Commands

Ping the ESP:

```
ros2 service call /audix/esp/ping std_srvs/srv/Trigger {}
```

Initialize/calibrate IMU:

```
ros2 service call /audix/esp/init_imu std_srvs/srv/Trigger {}
```

Reset odometry:

```
ros2 service call /audix/esp/reset_odom std_srvs/srv/Trigger {}
```

Stop immediately:

```
ros2 service call /audix/esp/stop std_srvs/srv/Trigger {}
```

Move forward 30 cm and wait for completion:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: 0.0,
distance_m: 0.30, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

Strafe right 20 cm and wait for completion:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: -90.0,
distance_m: 0.20, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

Strafe left 20 cm and wait for completion:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: 90.0,
distance_m: 0.20, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

Move backward 10 cm and wait for completion:

```
ros2 service call /audix/move audix_interfaces/srv/Move "{angle_deg: 180.0,
distance_m: 0.10, heading_deg: 0.0, timeout_s: 10.0, wait_for_done: true}"
```

Send manual twist RPM for 300 ms:

```
ros2 service call /audix/twist audix_interfaces/srv/TwistCommand "{forward_rpm:
20.0, strafe_rpm: 0.0, turn_rpm: 0.0, timeout_s: 0.3}"
```

Send a raw ESP command for debugging:

```
ros2 service call /audix/esp/raw_command audix_interfaces/srv/RawCommand "
{command: 'STATUS', timeout_s: 3.0, wait_for_done: false}"
```

Monitoring

Watch typed ESP telemetry:

```
ros2 topic echo /audix/esp/telemetry
```

Watch raw ESP JSON:

```
ros2 topic echo /audix/esp/telemetry_raw
```

Watch odometry:

```
ros2 topic echo /audix/odom
```

Watch IR sensors:

```
ros2 topic echo /audix/ir/state
```

Parameters

Common launch parameters:

```
port                 default /dev/ttyAMA0
baud                 default 115200
namespace            default audix
mock_ir              default false
ir_enabled           default true
ir_logic             default baseline
init_imu_on_start    default true
reset_odom_on_start  default true
```

Node-only parameters that can be set in a YAML file:

```
ack_timeout_s
move_timeout_s
telemetry_period_s
ir_poll_period_s
frame_id
base_frame_id
odom_forward_sign
odom_strafe_sign
reset_encoders_on_start
ir_active_low
ir_pull_up
verbose_serial
```

ROS odometry uses standard `x` forward, `y` lateral, yaw around `z`. If the robot appears mirrored in RViz, change `odom_strafe_sign` or `odom_forward_sign` instead of changing the ESP firmware.

Micro-ROS Migration Rule

Do not run micro-ROS Agent over the same UART while `esp_correct_pid_pi.ino` is using that UART for JSON commands.

The proper native micro-ROS ESP32 version would be a new firmware file with one ESP node, for example `/audix/esp32_controller`, exposing:

```
Publishers:
  /audix/esp/telemetry
  /audix/odom

Services or actions:
  /audix/move
  /audix/twist
  /audix/esp/stop
  /audix/esp/init_imu
  /audix/esp/reset_odom
```

That firmware would replace `Serial.println(JSON)` and command parsing with `rclc` publishers, services, timers, and an executor. The motor PID control loop should remain separate from the ROS executor so ROS traffic cannot stall motor timing.

Safe Test Order

1. Build the workspace.
2. Launch with `mock_ir:=true` first.
3. Call `/audix/esp/ping`.
4. Echo `/audix/esp/telemetry_raw`.
5. Call `/audix/esp/reset_odom`.
6. Call a short `/audix/move` command, such as 10 cm forward.
7. Launch without `mock_ir` and echo `/audix/ir/state`.
8. Test 20 cm forward/right/left/backward moves.